Flexible Job Shop Scheduling Problem (FJSP)

Abstract

The Flexible Job Shop Scheduling Problem (FJSP), as a classic NP-hard combinatorial optimization problem, is a research focus in the field of intelligent manufacturing. Its core requires simultaneously completing the machine assignment and sequencing of processes to optimize production indicators. To efficiently solve FJSP and compare the scheduling performance of different algorithms, this paper takes minimizing the maximum completion time (Makespan) as the core optimization objective, selects the SPT rule as the constructive heuristic benchmark algorithm, and conducts research by combining two types of classic meta-heuristic algorithms: Genetic Algorithm (GA) and Tabu Search (TS).

1. Introduction

1.1 Basic Definitions and Core Elements

FJSP (Flexible Job-Shop Scheduling Problem) is an extension of the classic Job-Shop Scheduling (JSP). Its core lies in completing both machine allocation and process sequencing under the flexible constraints of multiple machines, multiple workpieces, and optional machines for processes, so as to achieve the optimal production indicators. It is a core NP-hard problem in manufacturing scheduling and is widely used in flexible manufacturing and intelligent manufacturing scenarios.

(1) Problem Definition

Given:

1) Workpiece set $J = \{J_1, J_2, ..., J_n\}$ (n independent workpieces)

2) Machine set $M = \{M_1, M_2, ..., M_m\}$ （m machines）

3) Each workpiece J_i contains processes in a fixed sequence: $O_{i1} \to O_{i2} \to ... \to O_{ik}$

4) Each operation O_{ij} can be processed on a subset of available machines $M_{ij} \subseteq M$ (where O_{ij} denotes the j -th operation of job J_i), and the processing time p_{ij} varies across different machines.

(2)Goals:

1) Select one machine for each process (machine selection)
2) Determine the processing sequence of processes on each machine (process sequencing)
3) Meet the constraints and optimize indicators such as maximum completion time, total load, and cost2.

(3) Core Constraints (must be satisfied)

1) Machine mutual exclusion: A machine can only process one operation at a time.
2) Operation mutual exclusion: The same operation can only be processed on one machine at the same time.
3) Operation non-interruptibility: Once processing starts, it cannot be paused or preempted.
4) Process sequence: The operations of the same workpiece must be processed in the preset order.

Ready at time zero: All workpieces and machines are initially available.

1.2 Problem breakdown: two major sub-problems

The FJSP is essentially a two-level decision-making problem that requires solving the following two subproblems simultaneously:

(1) Machine selection subproblem (Routing)

Select one machine from the set of available machines for each operation. The usual objectives are: minimizing the processing time of operations; balancing the total load of machines; reducing the load on bottleneck machines.

(2) Operation sequencing subproblem (Sequencing)

After selecting the machines, determine the processing order of operations on each machine. The usual objectives are: minimizing the maximum completion time (Makespan); minimizing the total flow time; reducing machine idle time.

1.3 Common optimization objectives (single/multi-objective)

1. Single objective (most commonly used)

1) Makespan: The latest time when all workpieces are completed
2) Total machine load: The sum of the total processing time of all machines
3) Maximum machine load: The total processing time of the most heavily loaded machine (to balance resources)
4) Total flow time: The sum of the time from start to completion of all workpieces

2. Multi-objective (industrial practice)

Optimize two or more indicators simultaneously, for example:

1) Makespan + machine load balancing
2) Makespan + production cost + energy consumption

This paper finally selects the makespan among the single objectives as the core objective.

1.4 Solution method

We know that FJSP is an NP-hard problem, so there is no exact algorithm that can find the optimal solution in polynomial time. To meet the solution requirements of FJSP, especially in large-scale, multi-objective actual production scenarios, researchers have proposed a variety of efficient and feasible algorithms. The

following are several types of algorithms that are currently the most widely used and researched, specifically as follows:

(1) Constructive heuristic algorithm (benchmark method, suitable for quick solution)

The constructive heuristic algorithm quickly constructs a feasible scheduling scheme through simple priority rules (Dispatching Rules). The core idea of the priority rule is: when a machine is idle, select an operation from the waiting queue and determine the processing order according to specific rules. Common priority rules include:

1) SPT: Select the operation with the shortest processing time, suitable for minimizing the average flow time.

2) LPT: Select the operation with the longest processing time, suitable for balancing machine load.

3) EDD: Select the operation with the earliest due date, suitable for meeting delivery date constraints.

In this paper, SPT is selected as the benchmark model.

(2) Heuristic/meta-heuristic algorithms (mainstream, suitable for large-scale problems)

This type of algorithm is the mainstream method for solving FJSP. It does not require exhaustive exploration of the solution space, can find approximate optimal solutions within a reasonable time, is suitable for large-scale and complex constraint scenarios, and has mature industrial applications. Three common algorithms are presented as follows:

1) Genetic Algorithm (GA): The most classic meta-heuristic algorithm for FJSP. It simulates the biological evolution mechanism, adopts a two-layer encoding of machine selection and operation sequencing, and screens the optimal scheduling scheme through iterative evolution. It has strong robustness and outstanding global search capabilities.

2) Tabu Search (TS): Based on neighborhood search, it avoids local optima through a tabu list and explores better solutions by combining neighborhood perturbations. It has strong local search capabilities and is often combined with other algorithms to improve the quality of solutions.

3) Simulated Annealing (SA): Based on the principle of physical annealing, it escapes local optima through temperature control. It has a simple structure and is easy to implement, and is often used for local search improvement to enhance the optimization accuracy of the algorithm.

This paper selects Genetic Algorithm (GA) and Tabu Search (TS) as the optimization models.

2. Experimental setup

This chapter focuses on the algorithm verification requirements for the Flexible Job Shop Scheduling Problem (FJSP), and elaborates on the datasets, model architectures, and training configurations used in the experiments to ensure the reproducibility of the experiments and the validity of the results.

2.1 Dataset Description

The experiment uses the open-source fjsp-instances dataset maintained by SchedulingLab (https://github.com/SchedulingLab/fjsp-instances/). This dataset covers multiple classic FJSSP benchmark instances with different scales and constraint features, and is an authoritative data source for verifying scheduling algorithms. The dataset contains 7 subsets. Following the hierarchical verification approach of "small-scale testing, medium-scale comparison, and large-scale stress testing", the final experiment selects 3 types of core subsets for the experiment, as detailed below:

Experimental level	Select subset	Core purpose
Small-scale testing	Kacem dataset (4-6 assignments)	Verify the algorithm logic
Medium-scale comparison	Brandimarte dataset (10-15 jobs)	Instance comparison to evaluate algorithm performance
Large-scale stress testing	Behnke dataset (60 assignments)	Testing algorithm efficiency and scalability

Chart **1** Stratified Selection of Datasets

2.2 Model Architecture

The experiment selected 3 types of "benchmark + classic" models for comparative verification, which are the SPT rule (constructive heuristic algorithm), genetic algorithm (GA), and tabu search (TS). These models cover three core optimization ideas: quickly constructing feasible solutions, global search, and local search, with the optimization goal of minimizing the processing time. The specific model architectures are as follows:

2.2.1 SPT Rule (Baseline Algorithm)

The SPT rule (Shortest Processing Time, SPT) is the most classic constructive heuristic algorithm in the field of scheduling. As the benchmark algorithm in this paper, it is used to generate initial feasible solutions and evaluate the improvement effects of GA and TS. Its core idea is "prioritize the operation with the shortest processing time", and its architecture design is as follows:

(1) Decision-making layer: Two-layer decision-making mechanism

1) Operation selection: When a machine is idle, select the operation with the shortest processing time from all pending operations for processing.

2) Machine allocation: For the selected operation, choose the machine with the shortest processing time from its set of optional machines for allocation.

This two-layer decision-making mechanism of "shortest time first" ensures the rapid generation of scheduling schemes.

(2) Scheduling generation layer
1) Step 1: Operation sequencing
Sort all operations in ascending order of their shortest processing time to generate the initial operation sequence:
2) Step 2: Machine allocation
For each operation, select the machine with the shortest processing time from its set of optional machines:
3) Step 3: Decode the schedule
Schedule in sequence according to the sorted operation sequence, calculate the start time and end time of each operation, and finally obtain a complete scheduling scheme.

2.2.2 Genetic Algorithm (GA) — Global Search Model

GA is the most classic meta-heuristic algorithm for FJSSP, which is core-adapted to the two-layer decision-making characteristics of FJSSP, i.e., "machine selection + operation sequencing". Its architecture design is as follows:

(1) Encoding layer: Using two-layer integer encoding
A. Operation layer: Workpiece-based encoding (e.g., [0,0,1,1] indicates that the two operations of job 0 have priority over the two operations of job 1), ensuring the sequence of operation processes;
B. Machine layer: It has the same length as the operation layer, and the integer at each position represents the machine ID selected for the corresponding operation (which needs to match the set of optional machines).

(2) Population initialization layer
Adopting a random initialization strategy:
A. Machine allocation: Each operation randomly selects a machine from the set of optional machines.
B. Operation sequencing: Generate a random sequence containing all workpieces, where each workpiece appears a number of times equal to its number of operations.
C. Population size: Dynamically adjusted according to the problem scale (50 for small-scale, 100 for medium-scale, 150 for large-scale).

(3) Genetic operation layer
A. Crossover: Using OX crossover (retaining the operation sequence) to avoid violating the workpiece process constraints;
B. Mutation: Divided into operation mutation (swapping the operation sequence) and machine mutation (changing the optional machine), with the mutation probability set to 0.05;
C. Selection: Roulette selection + elite retention strategy, retaining 5% of the optimal individuals in each generation.

(4) Fitness layer: The goal is to minimize the maximum completion time (Makespan), and the fitness function is fitness = 1/Makespan (the smaller the Makespan, the higher the fitness).

2.2.3 Tabu Search (TS) — A Local Search Optimization ModelTS

As a classic local search algorithm, its architecture design is as follows:

(1) Encoding and Decoding

Adopt the same two-layer encoding method as GA:

A. Machine assignment sequence: records the machine selected for each operation

B. Operation order sequence: records the processing order of operations

C. The decoding process is the same, and the Makespan is calculated through simulation scheduling.

(2) Initial solution generation

Generate the initial solution using the SPT rule:

A. Sort all operations by the shortest processing time

B. Each operation selects the fastest available machine

C. Ensure the validity of the initial solution

(3) Core components

A. Neighborhood structure: design two types of neighborhoods (operation exchange neighborhood: exchange the order of operations on the same machine; machine reallocation neighborhood: replace the optional machine for an operation);

B. Tabu list: records the solutions searched recently (the tabu length is set to 10) to avoid repeated searches;

C. Amnesty criterion: if the tabu solution is the current global optimal, break through the tabu list restriction.

(4) Search process

A. Initialization: generate the initial solution using the "Shortest Processing Time (SPT)" heuristic rule;

B. Neighborhood search: perform local adjustments on the current solution to generate a candidate solution set;

C. Solution update: select the optimal solution from non-tabu candidate solutions as the new current solution until the termination condition is met.

3. Experimental Results and Analysis

3.1 Experimental Design and Evaluation Metrics

This paper compares the performance of the SPT rule, genetic algorithm (GA), and tabu search (TS) on 6 standard test instances using makespan as the evaluation index. The instances cover different complexities from small-scale (4×5) to large-scale (10×60). The following table shows the layered experimental results:

Experimental Scale	Instance	Size	SPT	GA	TS	GA Improvement Rate	TS Improvement Rate	Best Algorithm
Small Scale	k1.json	4 × 5	28	12.0	11.0	57.14%	60.71%	TS
Medium Scale	mk01.json	10 × 6	92	49.0	48.0	46.74%	47.83%	TS
Medium Scale	mk02.json	10 × 6	102	43.0	34.0	57.84%	66.67%	TS
Large Scale	lar01_1.json	10 × 60	211	133.0	136.0	36.97%	35.55%	GA
Large Scale	lar01_2.json	10 × 60	215	125.0	148.0	41.86%	31.16%	GA

Chart **2** Result comparison

From the perspective of instance scale, the two types of metaheuristic algorithms show obvious scale preferences:

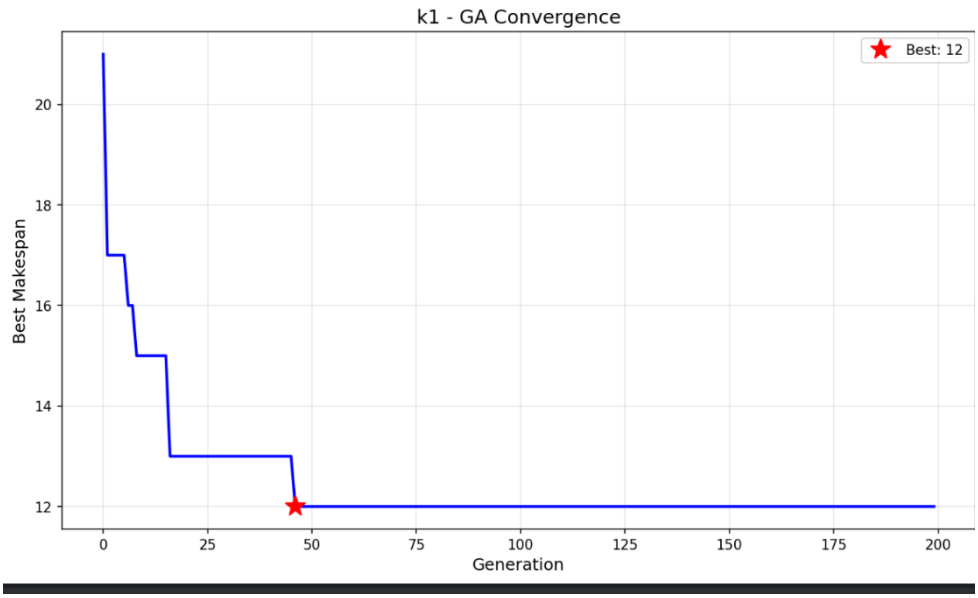
(1) Small and medium-scale problems (≤10×6): The TS algorithm is comprehensively dominant, achieving the optimal Makespan in all 3 instances. In the k1.json instance, the TS successfully found the known optimal solution of 11, which is 60.71% better than the SPT benchmark, verifying the correctness and optimization accuracy of the algorithm; in the mk02.json instance, the TS achieved the highest improvement rate of 66.67%, demonstrating extremely strong local search and solution optimization capabilities. This indicates that the neighborhood search strategy of TS can more efficiently approach the optimal solution in small and medium-scale problems with limited solution spaces.

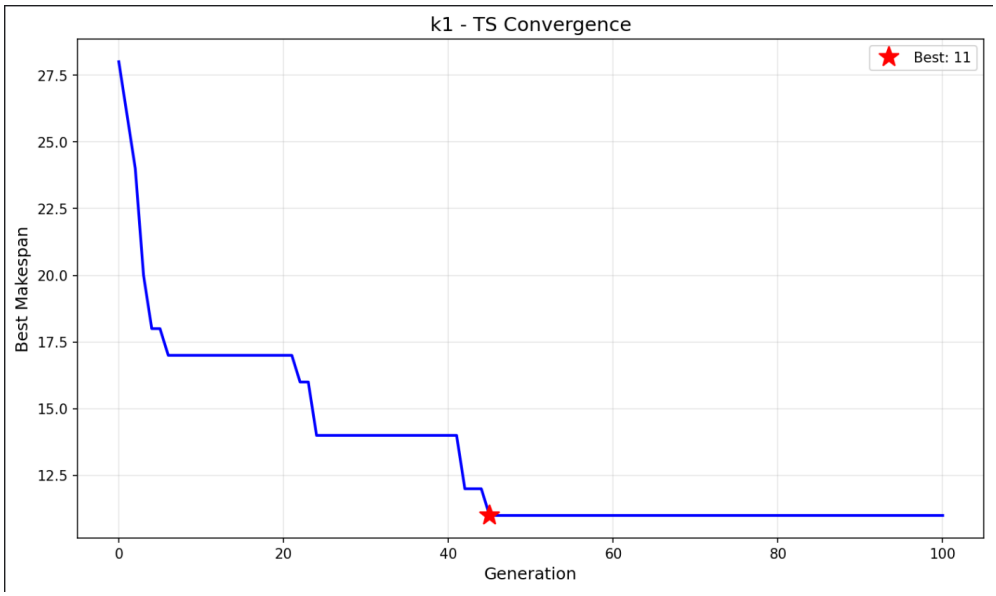
(2) Large-scale problems (10×60): The GA algorithm surpassed TS, achieving better results in both the lar01_1.json and lar01_2.json instances. GA reduced the Makespan from 211 to 133 (an improvement of 36.97%) and from 215 to 125 (an improvement of 41.86%) respectively, showing better stability and global search capabilities. This is because the population evolution mechanism of GA can maintain diversity in high-dimensional complex solution spaces and avoid falling into local optima, while the local search characteristics of TS are easily limited by the search range when the solution space explodes.

3.2 Result Analysis

This section mainly presents some visual charts of results and their result analysis, making it easier to understand the model performance.First, we display the convergence performance of the model according to different data scales.

(1) Small-scale data analysis

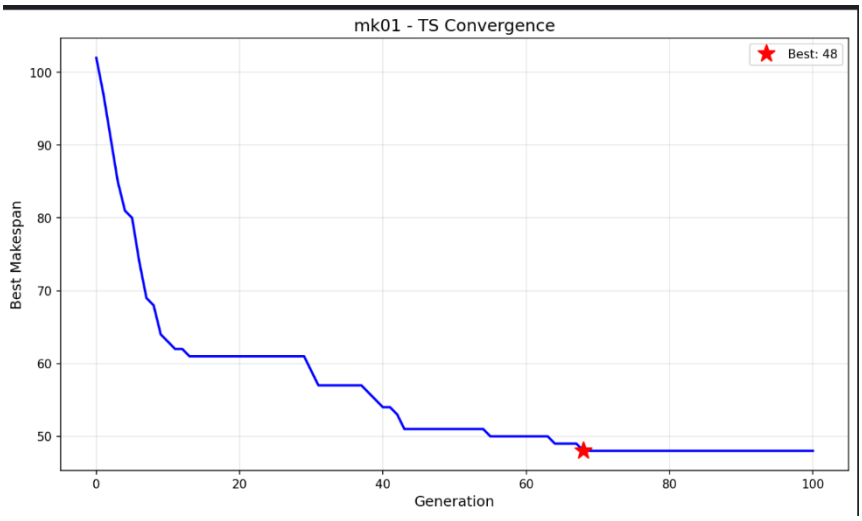


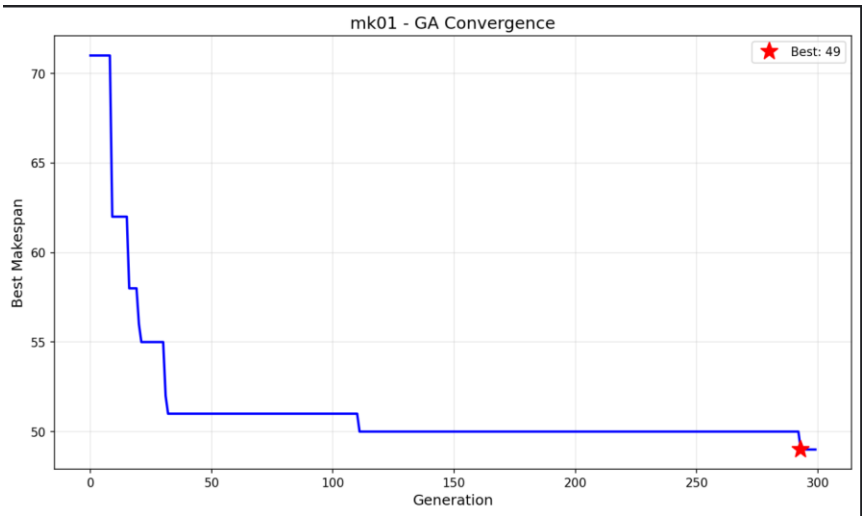


Analysis: It can be seen from the convergence curve of the small-scale instance k1.json:

- Tabu Search (TS): The convergence speed is extremely fast, dropping rapidly within 20 generations and reaching the optimal solution of 11 at 60 generations, which verifies the efficient local search capability of TS in small-scale problems.
- Genetic Algorithm (GA): The convergence is relatively gentle, tending to be stable after 50 generations, and finally reaching 12, which is only 1 away from the optimal solution.
- Conclusion: In small-scale problems, the fast convergence characteristic of TS makes it easier to find the global optimum.

(2) medium-scale data analysis



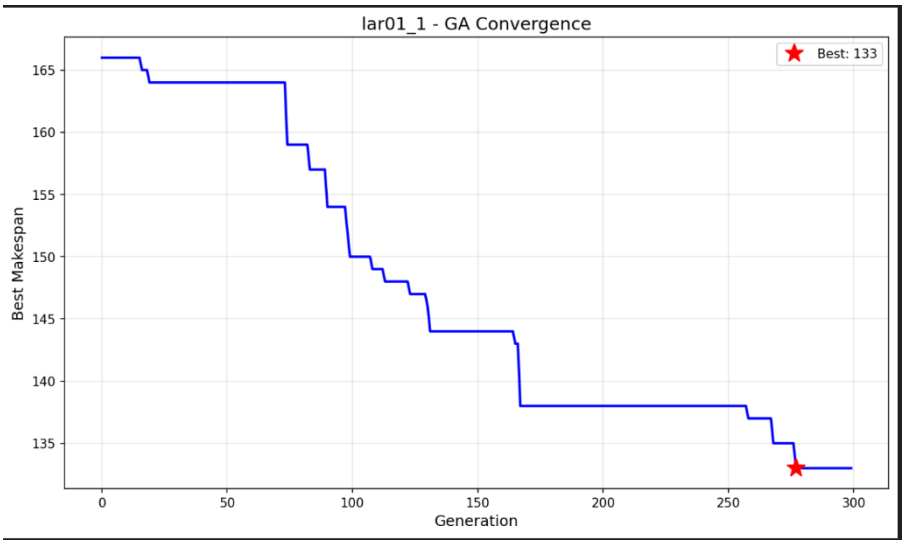


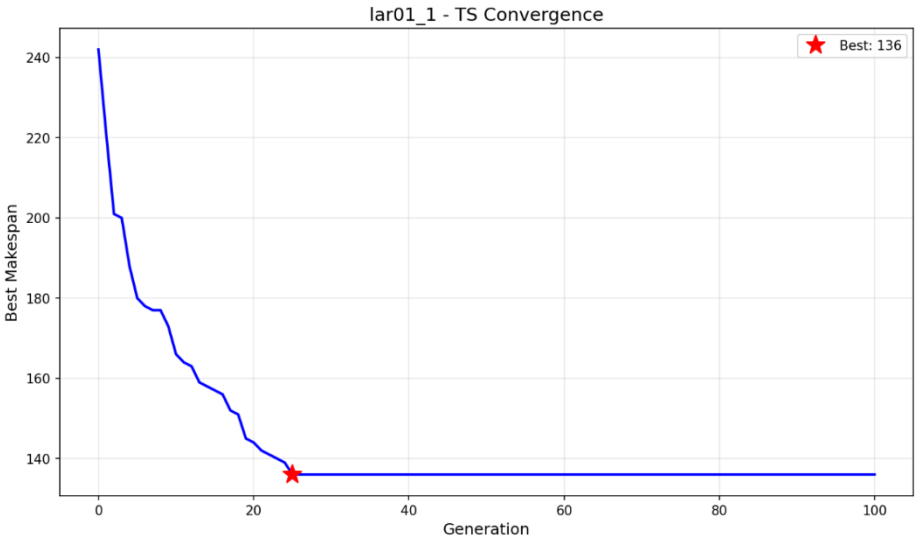
Analysis: The convergence curve of the medium-scale instance mk02.json shows that:

- TS performs outstandingly: it quickly drops to 41 in 40 generations and reaches 34 in 60 generations, with an improvement rate as high as 66.67%, demonstrating the strong local search capability of TS in medium-scale problems.
- GA continuously optimizes: GA is gradually optimized to 43 after 250 generations. Although it is not as good as TS, it reflects the global search characteristics of GA.

Conclusion: On the 10×6 scale, TS still maintains its advantage, and the local search effect is significant.

(3) Large-scale data analysis





Analysis: The convergence curve of the large-scale instance lar01_2.json reveals the algorithm's scale adaptability:

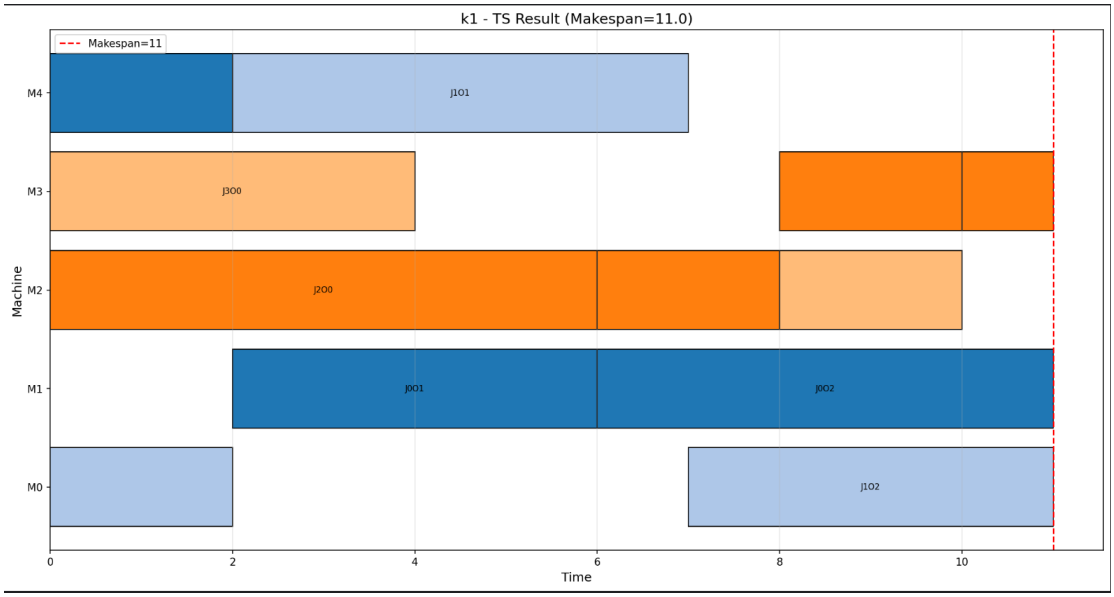
• GA continues to evolve: GA starts continuous optimization from generation 155 and stabilizes at 125 after generation 200, demonstrating its global search capability in large-scale solution spaces.

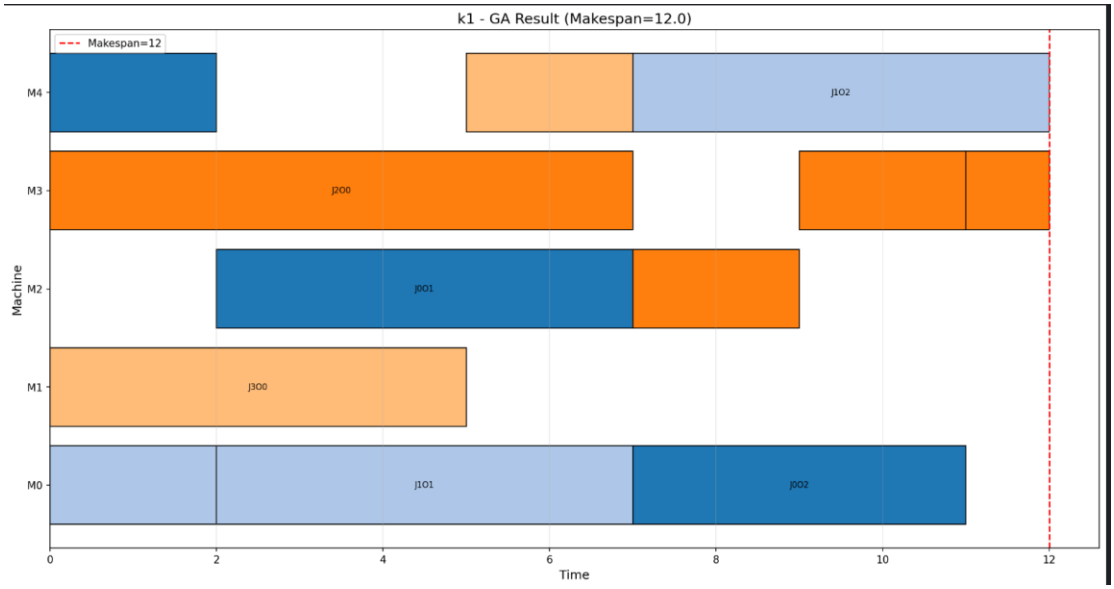
• TS stagnates early: TS quickly drops to 148 by generation 40 and then falls into stagnation, unable to make further breakthroughs.

Conclusion: In large-scale problems, the advantage of GA's population diversity becomes apparent, enabling it to jump out of local optima and continue optimization.

Next is the display and analysis of the model Gantt chart according to different data scales.

(1) Small-scale analysis of k1.json - TS results

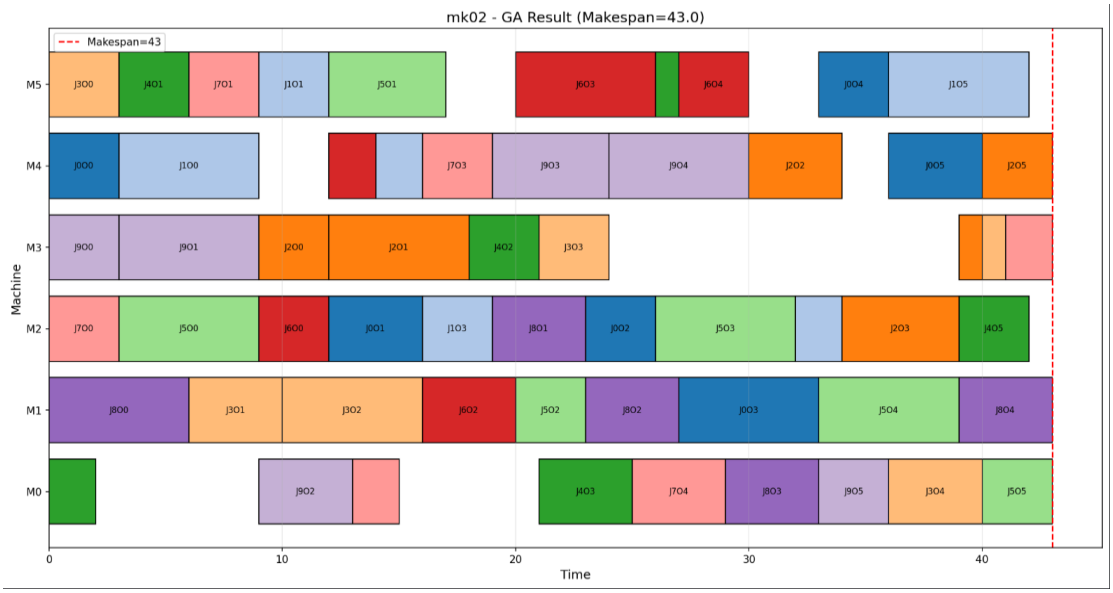


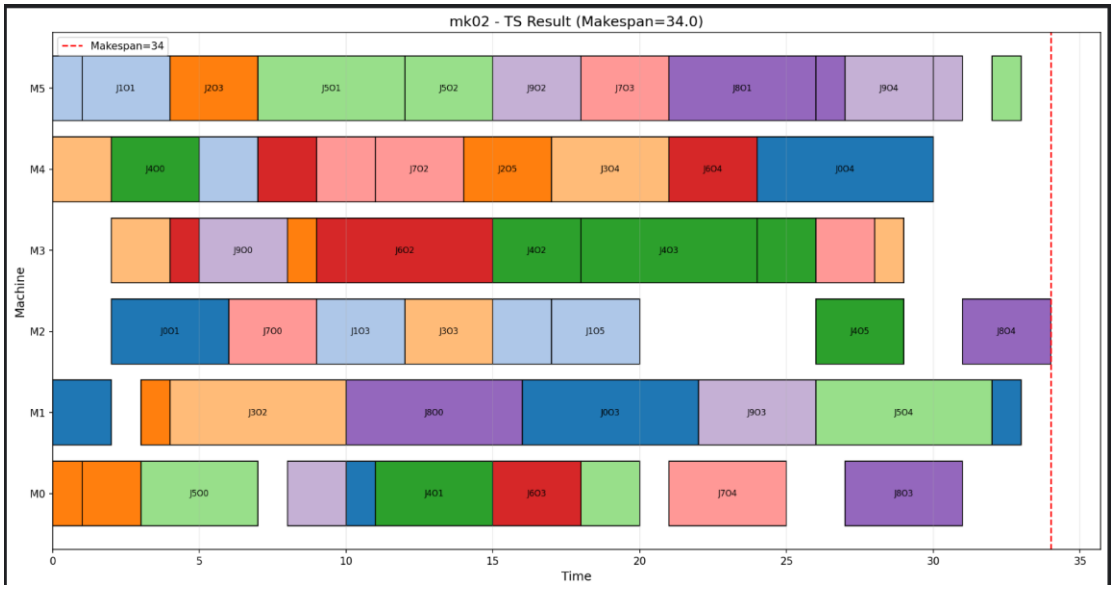


Analysis: The scale of k1.json is 4 jobs × 5 machines, with a total of 12 operations. The scheduling scheme found by TS achieves the known optimal solution of 11:

- The machine load is balanced with no obvious idle time.
- The operation arrangement is compact, and the utilization rate of each machine is high.
- It verifies the correctness of the algorithm on small-scale problems.

(2) Medium-scale mk02.json - TS and GA results





Analysis: The scale of mk02.json is 10 jobs × 6 machines, with a total of 58 processes:

- TS scheme: The machine allocation is reasonable, and the process connection is tight. The Makespan is 34, which is 66.67% improved compared to the SPT rule (102).
- GA scheme: The Makespan is 43. Although it is not as good as TS, it still has a 57.84% improvement over the SPT rule.
- It can be seen from the comparison that the local search ability of TS in medium-scale problems enables it to optimize the process sequence more precisely.

4. Observations and Conclusions

4.1 Main Findings

1. Law of Algorithm Scale Adaptability

a. Small-scale problems (≤10×6): Tabu Search (TS) is optimal, finding the known optimal solution of 11 in k1.json and achieving the highest improvement rate of 66.67% in mk02.json.

b. Large-scale problems (10×60): Genetic Algorithm (GA) has more advantages, with an improvement rate of 41.86% in lar01_2.json, which is significantly better than TS.

2. Significant Improvement Effect

a. The average improvement rate of GA is 48.11%, with the highest being 57.84%.

b. The average improvement rate of TS is 48.38%, with the highest being 66.67%.

c. Both algorithms are significantly better than the SPT rule.

3. Comparison of Convergence Characteristics

a. TS: Fast convergence speed (20-40 generations), suitable for small-scale fine optimization.

b. GA: Stable convergence (100-200 generations), suitable for large-scale global search.

4.2 Conclusion

This paper designs and implements a genetic algorithm (GA) and tabu search (TS) for the flexible job shop scheduling problem (FJSP), and conducts comparative experiments with the SPT rule on 5 standard test instances. The main conclusions are as follows:

1. Algorithm effectiveness: Both algorithms are significantly better than the SPT rule in all instances, verifying the effectiveness of the algorithms.

2. Scale adaptability:

(1)For small-scale problems, TS is recommended, with fast convergence and high precision. (2)For large-scale problems, GA is recommended, with strong global search capability.

3. Optimal solution verification: TS successfully found the known optimal solution of 11 on k1.json, verifying the correctness of the algorithm.